Neural Network From Scratch

Nagesh

May 2026

"ohhh ! here , there ... hahaha , everywhere , IT IS THE SAME !!!!!!!!!!!!" (Nagesh)

Contents

Introduction to Neural Networks **3**
- Artificial Neuron . . . 3
- Why Activation Functions? . . . 3

Universal Approximation Theorem **4**
- Theorem Statement . . . 4
- Interpretation . . . 4

Matrix Representation of Neural Networks **4**

Activation Functions **5**
- Sigmoid . . . 5
- Tanh . . . 5
- ReLU . . . 5
- Leaky ReLU . . . 6
- Softmax . . . 6

Loss Functions **6**
- Mean Squared Error . . . 6
- Binary Cross Entropy . . . 6
- Categorical Cross Entropy . . . 6

Backpropagation **6**
- Forward Pass . . . 6
- Backward Pass . . . 7

Optimization Algorithms **7**
- Gradient Descent . . . 7
- Momentum . . . 7
- AdaGrad . . . 7

RMSProp 7
Adam 8

Regularization Techniques **8**
L2 Regularization 8
L1 Regularization 8
Dropout 8

NumPy Broadcasting and Summation **9**

Simple Neural Network From Scratch **9**

Conclusion **9**

Introduction to Neural Networks

Neural networks are computational models inspired by biological neurons.
They learn complex nonlinear mappings directly from data.
Modern deep learning systems such as:

- Image recognition
- Large Language Models (LLMs)
- Speech recognition
- Protein folding
- Generative AI

are built using neural networks.

Artificial Neuron

A neuron computes:

$$z = w^T x + b$$

followed by an activation:

$$a = f(z)$$

where:

- x = input vector
- w = weight vector
- b = bias
- f = activation function

Why Activation Functions?

Without nonlinear activations:

$$f(x) = Wx + b$$

multiple layers collapse into a single linear transformation.
Nonlinear activations allow neural networks to learn complex functions.

Universal Approximation Theorem

A feedforward neural network with:

- one hidden layer
- sufficiently many neurons
- nonlinear activation function

can approximate any continuous function on a compact domain.

Theorem Statement

Given:

$$f(x) : [0,1]^n \to \mathbb{R}$$

continuous, for every:

$$\epsilon > 0$$

there exists a neural network:

$$F(x) = \sum_{i=1}^{N} \alpha_i \sigma(w_i^T x + b_i)$$

such that:

$$|f(x) - F(x)| < \epsilon$$

for all x in the compact domain.

Interpretation

Neural networks are universal function approximators.

Matrix Representation of Neural Networks

Suppose:

- Batch size $= m$
- Input dimension $= d$
- Hidden neurons $= h$
- Output classes $= c$

Then:

$$X \in \mathbb{R}^{m \times d}$$
$$W^{[1]} \in \mathbb{R}^{d \times h}$$
$$b^{[1]} \in \mathbb{R}^{1 \times h}$$

Forward propagation:

$$Z^{[1]} = XW^{[1]} + b^{[1]}$$

$$A^{[1]} = f(Z^{[1]})$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$$

Activation Functions

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Tanh

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Derivative:

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$

ReLU

$$f(x) = \max(0, x)$$

Derivative:

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Leaky ReLU

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

Derivative:

$$f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$$

Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Loss Functions

Mean Squared Error

$$L = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$

Binary Cross Entropy

$$L = -\frac{1}{N} \sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Categorical Cross Entropy

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Backpropagation

Backpropagation computes gradients using the chain rule.

Forward Pass

$$Z^{[1]} = XW^{[1]} + b^{[1]}$$

$$A^{[1]} = f(Z^{[1]})$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$$

$$\hat{Y} = \text{softmax}(Z^{[2]})$$

Backward Pass

For softmax + cross entropy:

$$dZ^{[2]} = \hat{Y} - Y$$

$$dW^{[2]} = (A^{[1]})^T dZ^{[2]}$$

$$db^{[2]} = \sum dZ^{[2]}$$

$$dA^{[1]} = dZ^{[2]}(W^{[2]})^T$$

$$dZ^{[1]} = dA^{[1]} \odot f'(Z^{[1]})$$

$$dW^{[1]} = X^T dZ^{[1]}$$

$$db^{[1]} = \sum dZ^{[1]}$$

Optimization Algorithms

Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta)$$

Momentum

$$v_t = \beta v_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

AdaGrad

$$G_t = G_{t-1} + g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$$

RMSProp

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t} + \epsilon} g_t$$

Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Regularization Techniques

L2 Regularization

$$L = L_{data} + \lambda \sum_i w_i^2$$

Gradient:

$$\frac{\partial L}{\partial w} = \frac{\partial L_{data}}{\partial w} + 2\lambda w$$

L1 Regularization

$$L = L_{data} + \lambda \sum_i |w_i|$$

Gradient:

$$\frac{\partial L}{\partial w} = \frac{\partial L_{data}}{\partial w} + \lambda \text{sign}(w)$$

Dropout

Randomly disables neurons during training.

Mask:

$$M_i \sim \text{Bernoulli}(p)$$

Output:

$$A_{drop} = \frac{M \odot A}{p}$$

NumPy Broadcasting and Summation

Given:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Column-wise sum:

$$\text{sum}(A, \text{axis} = 0) = [12, 15, 18]$$

Row-wise sum:

$$\text{sum}(A, \text{axis} = 1) = [6, 15, 24]$$

Broadcasting allows arrays with compatible shapes to interact automatically.

Simple Neural Network From Scratch

[language=Python] import numpy as np
X = np.array([[1.0, 2.0], [3.0, 4.0]])
W1 = np.random.randn(2,3) b1 = np.zeros((1,3))
Z1 = X @ W1 + b1 A1 = np.maximum(0,Z1)
W2 = np.random.randn(3,2) b2 = np.zeros((1,2))
Z2 = A1 @ W2 + b2

Conclusion

Neural networks are universal nonlinear function approximators.
Modern deep learning combines:

- nonlinear activations
- matrix computation
- backpropagation
- optimization algorithms
- regularization

to learn highly complex representations from data.